

Parametri e valori di ritorno

Parametri e valori di ritorno

Come passare dati a un metodo e ricevere un risultato.

Rendere un metodo piu flessibile

Un metodo senza parametri fa sempre la stessa cosa.

```
public static void saluta() {  
    System.out.println("Ciao, Luca!");  
}
```

Se vogliamo salutare nomi diversi, usiamo un **parametro**.

Parametri

```
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome + "!");  
}
```

`String nome` è un parametro: una variabile che il metodo riceve quando viene chiamato.

```
saluta("Luca");  
saluta("Sara");
```

Output:

```
Ciao, Luca!  
Ciao, Sara!
```

I valori passati al metodo si chiamano **argomenti**.

Piu parametri

Un metodo puo ricevere piu parametri.

```
public static void mostraProdotto(String nome, double prezzo) {  
    System.out.println(nome + ": " + prezzo + " euro");  
}
```

Chiamata:

```
mostraProdotto("Pane", 1.50);  
mostraProdotto("Latte", 1.20);
```

Output:

```
Pane: 1.5 euro  
Latte: 1.2 euro
```

L'ordine degli argomenti deve corrispondere all'ordine dei parametri.

void

Finora abbiamo usato `void` .

```
public static void saluta(String nome)
```

`void` significa: questo metodo non restituisce un valore.

Il metodo fa qualcosa, per esempio stampa un messaggio, ma non consegna un risultato a chi lo chiama.

return

Se un metodo deve produrre un risultato, devi indicare il tipo del valore restituito.

```
public static double calcolaTotale(double prezzo, int quantita) {  
    return prezzo * quantita;  
}
```

Qui il metodo restituisce un `double`.

Puoi usare il risultato così:

```
double totale = calcolaTotale(2.50, 4);  
System.out.println(totale);
```

Output:

```
10.0
```

Il tipo di ritorno deve corrispondere

Se dichiari che un metodo restituisce `int`, deve restituire un `int`.

```
public static int doppio(int numero) {  
    return numero * 2;  
}
```

Questo invece è sbagliato:

```
public static int saluto() {  
    return "Ciao"; // errore  
}
```

Il metodo promette un `int`, ma prova a restituire una `String`.

Esempio completo

```
public class Calcoli {  
    public static double calcolaTotale(double prezzo, int quantita) {  
        return prezzo * quantita;  
    }  
  
    public static void mostraTotale(String prodotto, double totale) {  
        System.out.println(prodotto + ": " + totale + " euro");  
    }  
  
    public static void main(String[] args) {  
        double totalePane = calcolaTotale(1.50, 2);  
        double totaleLatte = calcolaTotale(1.20, 3);  
  
        mostraTotale("Pane", totalePane);  
        mostraTotale("Latte", totaleLatte);  
    }  
}
```

Output:

```
Pane: 3.0 euro  
Latte: 3.5999999999999996 euro
```

Il secondo numero mostra una piccola imprecisione tipica dei numeri decimali nei computer. Per ora ci basta sapere che può succedere.

Regola pratica

Usa parametri quando il metodo ha bisogno di dati dall'esterno.

Usa `return` quando il metodo deve calcolare o preparare un valore da usare dopo.